Projet Wazuh – Détection d'intrusions

Informations

Étudiante : Amal LABIDI
Module : B2 Sécurité Pratique – TP1 / TP2
Détections : Déni de service HTTP (T1499), injection SQL (T1190), balayage de ports (T1046)
Plateforme : Wazuh 4.14 (manager + dashboard), machines Debian

1. Contexte et objectif

À partir de la plateforme Wazuh des TP1 et TP2, je mets en place **trois détections d'intrusion**. Pour chacune : configuration, attaque depuis la machine de test, puis lecture de l'alerte dans le dashboard.

Détection	MITRE	Règle d'alerte	Réponse active
Déni de service HTTP	T1499	100201	Oui (blocage IP)
Injection SQL	T1190	31103 / 31106	Non
Balayage de ports Nmap	T1046	100301	Non

ATTENTION — Cadre légal : Tests réalisés **uniquement sur mes machines de laboratoire**, dans un **réseau isolé**.
Accéder frauduleusement à un système d'information est un délit (**art. 323-1 et suivants du Code pénal**).

2. Schéma d'architecture

Machine	Rôle	Adresse IP
Serveur Wazuh	Manager + dashboard (règles, alertes)	192.168.213.157
Agent client1	Victime : Apache + agent Wazuh	192.168.213.134
Attaquante	Lance ab, curl, nmap	192.168.213.159

[Capture à insérer]
Capture A — Schéma réseau des 3 machines avec leurs IP (à réaliser)

3. Préparation du lab

- Snapshot des 3 VM avant les tests.
- Réseau isolé (host-only), sans accès Internet.
- **Règle d'or : l'agent collecte les logs, le serveur porte les règles.**

Vérifier que l'agent est connecté (sur le serveur) :

/var/ossec/bin/agent_control -l

[Capture à insérer]
Capture B — Agent client1 (ID 003) « active » dans le dashboard (envoyée, à renvoyer en fichier)

4. Détection 1 — Déni de service HTTP (MITRE T1499)

Explication

L'attaquant envoie un très grand nombre de requêtes pour saturer le serveur web. On alerte quand une même IP dépasse **100 requêtes en 10 secondes**, puis on la bloque.

Procédure

Étape 1 : Sur l'agent, lire le journal Apache (`/var/ossec/etc/ossec.conf`) :

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>
```

Étape 2 : Sur le serveur, ajouter les règles (`/var/ossec/etc/rules/local_rules.xml`) :

```
<group name="local,web,dos,">
  <rule id="100200" level="3">
    <if_sid>31108</if_sid>
    <description>Requete web recue (base flood)</description>
  </rule>
  <rule id="100201" level="10" frequency="100" timeframe="10">
    <if_matched_sid>100200</if_matched_sid>
    <same_source_ip />
    <description>DoS HTTP : plus de 100 requetes en 10 s</description>
    <mitre><id>T1499</id></mitre>
  </rule>
</group>
```

Étape 3 : Sur le serveur, activer la réponse active (`/var/ossec/etc/ossec.conf`) :

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100201</rules_id>
  <timeout>180</timeout>
</active-response>
```

Étape 4 : Redémarrer le manager :

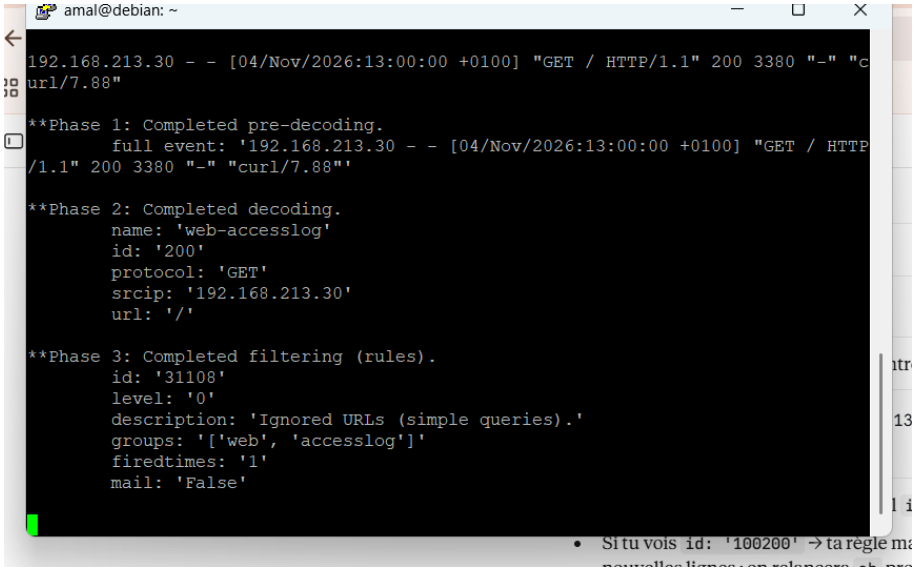
systemctl restart wazuh-manager

```
EOF
root@debian:~# systemctl restart wazuh-manager
grep -n "100200" /var/ossec/etc/rules/local_rules.xml
^C^C
32:  <rule id="100200" level="3">
37:    <if_matched_sid>100200</if_matched_sid>
root@debian:~# systemctl restart wazuh-manager
root@debian:~# grep -n "100200" /var/ossec/etc/rules/local_rules.xml
32:  <rule id="100200" level="3">
37:    <if_matched_sid>100200</if_matched_sid>
```

Règle 100200 chargée dans local_rules.xml

Avant

Une requête normale ne déclenche aucune alerte (règle 31108, niveau 0).



Avant : requête simple → règle 31108, niveau 0

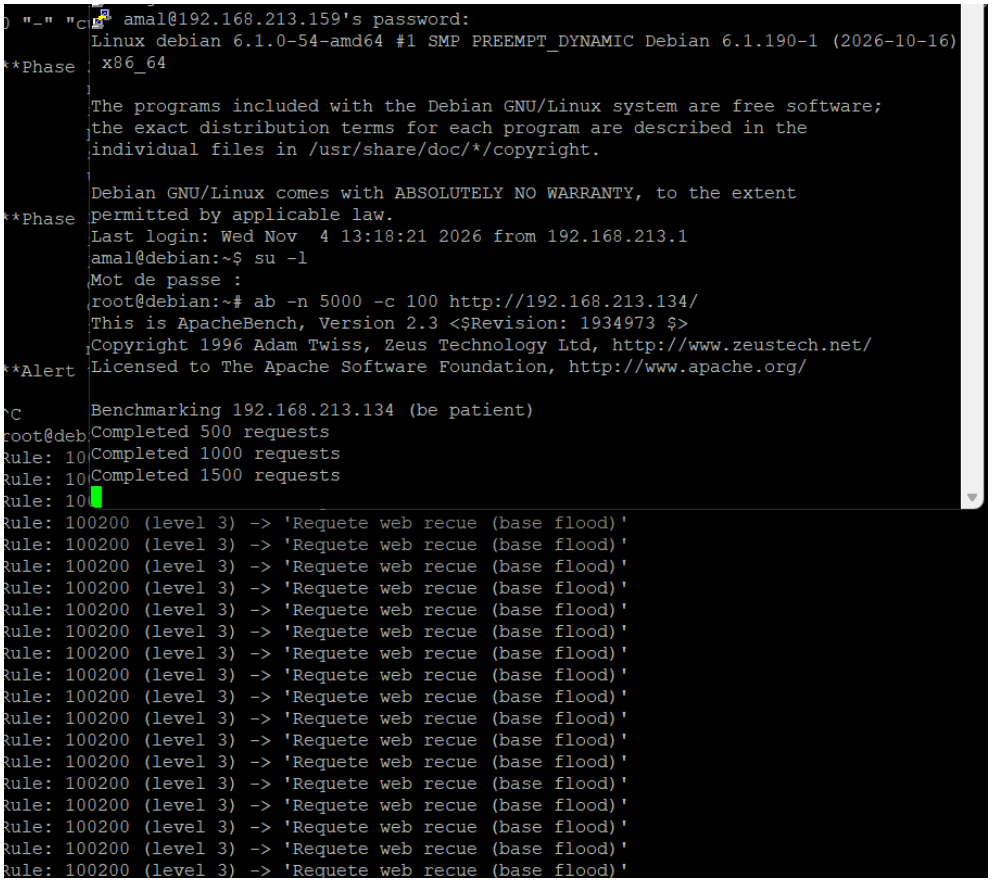
Pendant

Étape 5 : Depuis l'attaquante, lancer l'attaque :

ab -n 5000 -c 100 http://192.168.213.134/

Étape 6 : Sur le serveur, suivre les alertes :

tail -f /var/ossec/logs/alerts/alerts.log | grep -E "100200|100201"

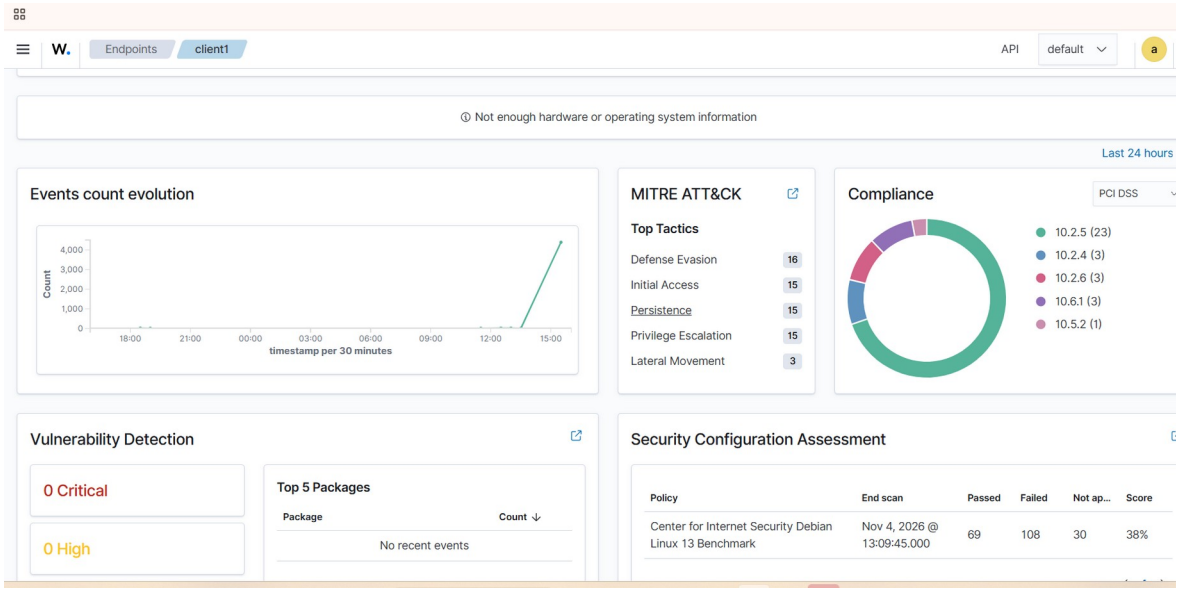


Pendant : ab en cours, les alertes défilent sur le serveur

[Capture à insérer]

Capture C — Alerte 100201 (niveau 10, T1499) dans le dashboard (à réaliser)

Après



Après : pic d'environ 4 400 événements sur client1

[Capture à insérer]

Capture D — Threat Hunting client1 : 4 418 alertes, groupes attack / web / dos (envoyée, à renvoyer en fichier)

[Capture à insérer]

Capture E — Alerte 651 « Host Blocked by firewall-drop » dans le dashboard (à réaliser)

Vérifier le blocage sur l'agent :

iptables -L INPUT -n | grep 192.168.213.159

[Capture à insérer]

Capture F — Règle DROP pour 192.168.213.159 dans iptables sur l'agent (à réaliser)

Analyse

Règle **100201**, niveau **10**, MITRE **T1499**, IP source **192.168.213.159**. Elle déclenche `firewall-drop` : l'IP est bloquée 180 s (alerte **651**).

5. Détection 2 — Injection SQL (MITRE T1190)

Explication

L'attaquant place du code SQL dans un paramètre d'URL pour interroger la base de données. Wazuh le détecte **nativement** : aucune configuration à ajouter.

Procédure

Étape 1 : Depuis l'attaquante, envoyer la requête :

curl -XGET "http://192.168.213.134/users/?id=SELECT+*+FROM+users"

[Capture à insérer]

Capture G — Commande curl et réponse 404 Not Found (envoyée, à renvoyer en fichier)

Étape 2 : Dans le dashboard (Threat Hunting), filtrer : `rule.id:31103`

[Capture à insérer]

Capture H — rule.id:31103 : 1 alerte, niveau 7, groupe sql_injection (envoyée, à renvoyer en fichier)

Analyse

Règle **31103** « SQL injection attempt », niveau **7**, MITRE **T1190**. La règle **31106** (niveau 12) ne s'est pas déclenchée car le serveur a répondu **404** : l'attaque est détectée mais n'a pas réussi.

6. Détection 3 — Balayage de ports Nmap (MITRE T1046)

Explication

L'attaquant teste de nombreux ports pour trouver les services ouverts. Le pare-feu de l'agent journalise chaque connexion ; on alerte au-delà de **15 en 5 secondes** depuis une même IP.

Procédure

Étape 1 : Sur l'agent, installer les outils et journaliser les connexions :

apt install iptables rsyslog
iptables -I INPUT -p tcp --syn -j LOG --log-prefix "SCAN-DETECT: "

Étape 2 : Sur l'agent, lire `kern.log` (`/var/ossec/etc/ossec.conf`), puis redémarrer l'agent :

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/kern.log</location>
</localfile>
```

Étape 3 : Sur le serveur, ajouter les règles (`local_rules.xml`) puis redémarrer le manager :

```
<group name="local,scan,">
  <rule id="100300" level="3">
    <if_sid>4100</if_sid>
    <match>SCAN-DETECT</match>
    <description>Connexion journalisee par iptables</description>
  </rule>
  <rule id="100301" level="10" frequency="15" timeframe="5">
    <if_matched_sid>100300</if_matched_sid>
    <same_source_ip />
    <description>Scan de ports detecte</description>
    <mitre><id>T1046</id></mitre>
  </rule>
</group>
```

Avant

[Capture à insérer]

Capture I — rule.id:100301 : « No results match your search criteria » (envoyée, à renvoyer en fichier)

Pendant

Étape 4 : Depuis l'attaquante, lancer le scan :

nmap -sT -p 1-1000 192.168.213.134

[Capture à insérer]

Capture J — Résultat nmap : ports 22 et 80 ouverts (envoyée, à renvoyer en fichier)

Après

[Capture à insérer]

Capture K — Alerte 100301 (niveau 10, T1046) dans le dashboard (à réaliser)

Analyse

Règle **100301**, niveau **10**, MITRE **T1046**, IP source **192.168.213.159**. 1000 ports testés en 0,24 s : le seuil est dépassé immédiatement.

7. Analyse des alertes

Règle	Niveau	Signification	MITRE
100201	10	DoS HTTP : > 100 requêtes / 10 s	T1499
31103 / 31106	7 / 12	Tentative d'injection SQL / attaque réussie (code 200)	T1190
100301	10	Scan de ports : > 15 connexions / 5 s	T1046
651	3	IP bloquée par la réponse active	—

[Capture à insérer]

Capture L — Vue globale Threat Hunting : 4 469 alertes, 0 niveau ≥ 12 (envoyée, à renvoyer en fichier)

8. Difficultés rencontrées et solutions

Difficulté	Solution
Pas d'Internet (réseau isolé)	NAT temporaire pour installer les paquets
Confusion agent / serveur	`wazuh-control info | grep type`
Règle DoS ignorée (web-accesslog est un décodeur, pas un groupe)	Utiliser `if_sid 31108`
L'agent ne remontait pas les logs	Redémarrer `wazuh-agent`
Manager planté (décodeur iptables inexistant + doublons)	Tester avec `wazuh-analysisd -t` avant de redémarrer
kern.log absent (journald, pas rsyslog)	Installer `rsyslog`
Règle scan sans résultat (la règle 4100 gagnait)	`if_sid 4100` + niveau 3

[Capture à insérer]

Capture M — Erreur « impossible d'ouvrir /var/log/kern.log » (envoyée, à renvoyer en fichier)

9. Limites et améliorations

- **Seuils fixes** : un scan lent ou un DDoS (plusieurs IP) passe sous le seuil → ajuster les seuils, ajouter une liste blanche.
- **Injection SQL** : un encodage différent ou une requête POST échappe à la règle → ajouter un WAF (ModSecurity).
- **Faux positifs** : un pic de trafic légitime peut bloquer un utilisateur → exclure les IP de confiance.
- **Amélioration** : étendre la réponse active au scan et envoyer les alertes par e-mail.

10. Conclusion

Les trois détections fonctionnent : le DoS est détecté et bloqué automatiquement, l'injection SQL est reconnue par les règles natives, et le scan Nmap devient visible grâce aux logs iptables. J'ai surtout retenu la séparation agent (collecte) / serveur (règles) et l'importance de tester la configuration avant de redémarrer.

11. Sources

- Documentation Wazuh : https://documentation.wazuh.com
- MITRE ATT&CK (T1499, T1190, T1046) : https://attack.mitre.org
- Code pénal, art. 323-1 : https://www.legifrance.gouv.fr
- Cours et sujet B2 Sécurité Pratique, A. Lopez